

ASSIGNMENT 1

PART D – RELATIONSHIP TABLE

Course: B.Tech. CSE

Semester: 5

Subject: System Design (TCS-504)

System: Movie Ticket Booking System

Submission Date: 07-September-2026

D. RELATIONSHIP TABLE

The relationships between classes are identified using the **lifetime test**. If the whole object is destroyed and the part should also be destroyed, the relationship is **composition**. If the part can exist independently, **aggregation or association** is used.

Pair	Relationship	Justification using Lifetime Test
Cinema — Screen	Composition	A Screen belongs to a particular Cinema and forms part of its structure. If the Cinema is destroyed, its Screens are no longer part of that Cinema.
Screen — Seat	Composition	Seats are physical seats belonging to a particular Screen. If the Screen is removed, its associated Seats no longer exist as part of that Screen.
Show — Movie	Aggregation	A Movie can exist independently of a Show. Multiple Shows can use the same Movie, so destroying a Show does not destroy the Movie.

Show — Screen	Association	A Show uses a Screen for a particular screening. The Screen can exist before, after, and independently of the Show.
Show — ShowSeat	Composition	ShowSeat represents the availability of a particular Seat for one specific Show. If the Show is destroyed, its ShowSeat objects are no longer required.
ShowSeat — Seat	Association	A ShowSeat refers to a physical Seat, but the Seat belongs to the Screen and can exist independently of any particular Show.
Booking — Customer	Association	A Booking is associated with a Customer, but the Customer can exist without a Booking and can create multiple bookings.
Booking — Show	Association	A Booking is made for a particular Show, but the Show exists independently of any particular Booking.
Booking — ShowSeat	Aggregation	A Booking refers to the ShowSeats selected by the Customer. The ShowSeats belong to the Show and can exist independently of a particular Booking.
Booking — Payment	Association	A Booking uses a Payment to complete the transaction, but the Payment object is not a permanent physical part of the Booking.
Payment — UpiPayment	Inheritance	UpiPayment is a specialized type of Payment and provides its own implementation of the <code>pay()</code> method.

Payment — CardPayment	Inheritance	CardPayment is a specialized type of Payment and provides its own implementation of the <code>pay()</code> method.
Payment — CashPayment	Inheritance	CashPayment is a specialized type of Payment and provides its own implementation of the <code>pay()</code> method.
BookingService — Booking	Association	BookingService creates and manages the booking process, but a Booking is a separate object with its own identity and can exist after the service operation.
BookingService — PriceCalculator	Association	BookingService uses PriceCalculator to calculate the total amount, but PriceCalculator can exist independently of a particular booking.
BookingService — TicketPrinter	Association	BookingService uses TicketPrinter to print the confirmed ticket. TicketPrinter is a separate utility and can exist independently of the booking service.
BookingService — Payment	Association	BookingService uses the Payment abstraction to process a customer's payment. The specific Payment object is independent of the BookingService.

1. Composition Relationships

The following relationships use **composition** because the part is strongly dependent on the whole:

Cinema ◆— Screen

A Cinema contains its Screens. The Screen is considered part of the Cinema's structure.

UML:

Cinema ◆—— Screen

Screen ◆— Seat

A Screen contains its physical Seats.

UML:

Screen ◆—— Seat

Show ◆— ShowSeat

Each Show creates/manages its own show-specific seat status.

UML:

Show ◆—— ShowSeat

2. Aggregation Relationships

Aggregation is used where the referenced object can continue to exist independently.

Show ◇— Movie

A Movie exists independently and can be used in multiple Shows.

UML:

Show ◇—— Movie

Booking ◇— ShowSeat

A Booking contains references to selected ShowSeats, but ShowSeats belong to the Show and are not permanently owned by a Booking.

UML:

Booking ◇———— ShowSeat

3. Association Relationships

The following are normal associations because the objects are connected but neither object owns the lifetime of the other:

Show ————— Screen

ShowSeat ————— Seat

Booking ————— Customer

Booking ————— Show

Booking ————— Payment

BookingService ————— Booking

BookingService ————— PriceCalculator

BookingService ————— TicketPrinter

BookingService ————— Payment

4. Inheritance Relationships

`Payment` is an abstract class. The three concrete payment classes inherit from it.

All three subclasses implement:

+ `pay(amount : double) : bool`

This allows `BookingService` to work with the general `Payment` type without depending on one specific payment method.

5. Final Relationship Summary

Relationship Type	Relationships
Composition	Cinema–Screen, Screen–Seat, Show–ShowSeat
Aggregation	Show–Movie, Booking–ShowSeat
Association	Show–Screen, ShowSeat–Seat, Booking–Customer, Booking–Show, Booking–Payment, BookingService–Booking, BookingService–PriceCalculator, BookingService–TicketPrinter, BookingService–Payment
Inheritance	Payment–UpiPayment, Payment–CardPayment, Payment–CashPayment

6. Conclusion

The relationship analysis defines how the classes in the Movie Ticket Booking System interact with each other. **Composition** is used where the part depends strongly on the lifetime of the whole, such as **Cinema-Screen**, **Screen-Seat**, and **Show-ShowSeat**. **Aggregation** is used where an object can exist independently, such as **Show-Movie** and **Booking-ShowSeat**. **Association** represents normal interaction between independent classes, while **inheritance** allows **UpiPayment**, **CardPayment**, and **CashPayment** to specialize the abstract **Payment** class.

This relationship structure is consistent with the class responsibilities and UML class diagram developed for the Movie Ticket Booking System.